Este ejercicio consiste en crear un programa usando las herramientas de java swing y todo lo conocido anteriormente en lo que se refiere a los ArrayList y los HashMap.

He creado un sistema que gesitona vehiculos y que contiene un CRUD (Copy, Read, Update, Delete), validacion, interfaz y listas dinámicas.

El programa divide las responsabilidades utilizando dos clases.

La clase Main. Que se encarga de gestionar el HashMap donde se almacenan los vehiculos, la ventana, los componentes y las funciones dentro de los listeners (luego lo explico) que darán como resultado una constante comunicación con el usuario a través de interfaces gráficas. El usuario se encarga de escoger las opciones que desee a través de esta interfaz.

Asi ha quedado la interfaz de mi app. Una ventana responsive con 5 botones clicables. Cada uno dispara una acción y hace cosas como almacenar, eliminar, buscar, mostrar o salir.

L**a clase Vehículo**. Esta clase se encarga de gestionar los atributos de la clase, los getters/setters, los métodos equals y hashcode, y el método toString (en este ejercicio he dado cuenta de cuan útil puede llegar a ser un toString bien configurado).

Empezaré por la clase Vehículo:

```
private String marca;
private String modelo;
private int cilindrada;
private ArrayList<String> complementos;
private double precio;
private int tipo;
private String matricula;
```

Esta clase tiene estos atributos.
El resto es lo de siempre. Ctr+i y configuramos constructores y getters/setters.
En esta ocasión utilicé varios constructores para ir haciendo pruebas distintas.

Voy a pasar directamente con los metodos equals() y hashcode(). —>

```java
@Override
public boolean equals(Object obj) {
    if (this == obj) {
        return true;
    }
    if (obj == null) {
        return false;
    }
    if (getClass() ≠ obj.getClass()) {
        return false;
    }

    Vehiculo other = (Vehiculo) obj;
    if (this.matricula == null) {
        if (other.matricula ≠ null) {
            return false;
        }
    } else if (!matricula.equals(other.matricula)) {
        return false;
    }
    return true;
}
```

Para este método me he basado en los apuntes de los imprimibles de Linkia. Se trata de comparar this con obj y luego definir sobré que atributo vamos hacer la comparación definitiva. En este caso si this.matricula no es como other.matricula entonces: false.

Si el método consigue pasar por todas las lineas y al final se cumple el último true. Los dos objetos son iguales.

```java
@Override
public int hashCode() {
    if (this.matricula == null) {
        return 0;
    }
    return this.matricula.hashCode();
}
```

Este método genera un número (hash) para el objeto usando la mátricula. Si es null (no hay matrícula, recuerda que usamos null porque estamos comprando un string), devuelve 0 para evitar errores. Si no, usa el hashCode() de la matricula, que es lo que identifica al objeto.

```java
@Override
public String toString() {
    String tipoTexto = (tipo ==1) ? "coche" : "moto";

    return "Vehiculo: " +
            "Marca: " + marca +
            " Modelo: " + modelo +
            " Cilindrada: " + cilindrada + "cc" +
            " Matricula: " + matricula+
            " Tipo: "+ tipoTexto +
            " Complementos: " + complementos;

}
```

Este método toString() tiene una lógica implementada para interactuar con el ArrayLIst. En una variable llamada tipoTexto guardo el siguiente resultado en forma de condicional ternario - Si el tipo es igual a 1 es (?) un coche, sino (:) es una moto. Así cuando ejecutemos la funcion mostrar nos enseñará si lo que ha escogido el usuario es una moto o un coche.

Ahora vamos con el metodo main que en este caso le he llamado ventanaPrincipal en honor a JavaSwing.

LOS IMPORTS:

```
package ACT_13;

import java.awt.Font;
import java.awt.GridLayout;
import java.util.ArrayList;
import java.util.HashMap;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JOptionPane;
```

- Font define el estilo y tamaño del texto.
-GridLayout organiza componentes en una cuadricula
- ArrayList. Lista dinámica de elementos
- HashMap. Alamacena pares clave_valor
- JButton. Crea botones en la interfaz
- JFrame crea la ventana principal
- JOption pane muestra diálogos de entrada y mensajes.

LA CLASE:

```
public class VentanaPral {

    public static void main(String[] args) {

        HashMap<String, Vehiculo> vehiculos = new HashMap();
        JFrame ventana = new JFrame();//crear ventana
        //configurar ventana
        ventana.setSize(500, 400);//tamaño de la ventana
        ventana.setTitle("Concesionario");//titulo de la ventana
        ventana.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);//se cierra el programa haciendo click en la x
        ventana.setLocationRelativeTo(null);//centrada
//        ventana.setLayout(null);//yo decido donde va a ir todo  ==> gridLayout lo pisa
        ventana.setLayout(new GridLayout(5,1));
```

- Declaración del HashMap vehiculos.
- Creacion de la ventana
- Configuración de la ventana:
  - setSize -> tamaño de la ventana
  - setTitle -> titulo de la ventana
  - setDefaultCloseOperation(Jframe.EXIT_ON_CLOSE) -> Cierra el programa haciendo click en la ‘x’ de la ventana.
  - setLayout -> reparte los elementos por la ventana. En este caso son 5 elementos en una ventana ==> ventana.setLayout(5,1);

```
//crear los componentes
JButton boton = new JButton("Ingresar Vehículo");
JButton botonMostrar = new JButton("Mostrar vehículos");
JButton botonEliminar = new JButton("Eliminar vehículo");
JButton botonBuscar = new JButton("Buscar vehículo");
JButton botonSalir = new JButton("Salir");
```

Bloque de crearcion de botones con JButton.

BLOQUE LISTENERS:

Los listeners son acciones que nos permiten ‘poner en marcha’ el mecanismo de los botones que nosotros hayamos implementado. La firma que he utilizado siempre ha sido la misma.

botton.addActionListener(e->{//logica programa…});

Voy a explicar el boton principal que he utilizado para ingresar datos en el programa.

```
// boton
boton.addActionListener(e -> {
    // 1. pedir datos
    String marca = JOptionPane.showInputDialog("Introduce la marca: ");
    String modelo = JOptionPane.showInputDialog("Introduce el modelo");
```

Después de la firma usamos esta sintaxis. JOptionPane es la herramienta que nos perimte que estos mensajes salgan por pantalla en forma de ventana emergente.

```
if (marca == null || modelo == null || marca.isEmpty() || modelo.isEmpty()) {/
    JOptionPane.showMessageDialog(null, "Error: campos vacios");
    return;
```

Patrón de comprobación de seguridad por si el hashmap está vacío. Es una buena práctica. De esta manera introducimos la marca del vehiculo. Como nos da igual que haya repeticiones en marca porque muchos vehiculos pueden tener la misma marca, lo dejamos así.

En la siguiente acción dentro del botón la cosa será diferente porque nos interesa controlar las excepciones ya que solo queremos que el usuario introduzca numeros.

```
int cilindrada;
while(true){
    String cilindradaStr = JOptionPane.showInputDialog("Intoduce la cilindrada");

    if(cilindradaStr == null || cilindradaStr.isEmpty()){
        JOptionPane.showMessageDialog(ventana, "campo vacío");
        continue;
    }

    try{
        cilindrada = Integer.parseInt(cilindradaStr);
        break; // salde del bucle si todo va bien
    }catch(NumberFormatException f){
        JOptionPane.showMessageDialog(ventana, "Introduce un número válido");
    }
```

En este caso tengo que utilizar un bucle para controlar estas excepciones.
Primero se pide al usuario que introduzca la cilindrada.
Se hace la comprobacion de seguridad por si el usuario no introduce nada y simplemente hace spam en aceptar. Hasta que el usuario no rellene el campo no puede pasar a la siguiente accion.

Seguidamente utilizo un bloque try/catch para controlar que el usuario introduzca numeros y no otro caracter. Dentro del try hago un cast de string a integer porque JOptionPane solo se mueve con Strings, así que es una práctica habitual a la hora de trabajar con esta herramienta. Si el usuario introduce otro carácter que no sea un número saltará el mensaje 'introduce un numero válido' (podría esmerarme más en estos mensajes).

Voy a explicar el bloque que controla el ArrayLIst dentro del boton.

```
double precio;
String precioStr = JOptionPane.showInputDialog("Intoduce el precio");
precio = Double.parseDouble(precioStr);
```

En este caso el cast lo hago justo después que el usuario introduzca el precio.

```
ArrayList<String>complementos = new ArrayList<>();
int pocion = JOptionPane.showConfirmDialog(ventana, "¿Quieres añadir complementos?");
if(pocion == JOptionPane.YES_OPTION){
    while(true){
        String complementoSrt = JOptionPane.showInputDialog("Introduce complemto");

        if(complementoSrt== null || complementoSrt.isEmpty()){
            JOptionPane.showMessageDialog(ventana, "Campo vacío");
            continue;
        }

        complementos.add(complementoSrt);

        int otro = JOptionPane.showConfirmDialog(ventana, "¿Añadir otro?");
        if(otro != JOptionPane.YES_OPTION){
            break;
        }
    }
}
```

Declaro un ArrayList con el nombre complementos para trabajar con el.
Pregunto al usuario si quiere añadir los complementos.
Con un condicional if -> utilizamos JOptionPane.YES_OPTION indica si el usuario ha pulsado la tecla 'si'. Si es así procedo a través de un bucle while a controlar la entrada del usuario. Esto lo hago así porque doy la posibilidad de introducir al usuario tantos complementos como el desee. Cada vez que introduce uno uso el metodo propio de ArrayList .add y pregunto si quiere añadir otro elemento más. Para esto uso un condicional if, si lo márcado es diferente a YES_OPTION salta el break y sale del bucle.

Una vez hemos pasado por todos los botones generamos la clave del HashMap en forma de matrícula en este caso.

```
//2. generar matrícula
String matricula = "B-" + vehiculos.size();
```

```
//3. crear un objeto con esos datos
Vehiculo v = new Vehiculo(marca,modelo,cilindrada,tipo,precio,matricula,complementos);
```

Generamos un objeto con estos datos (el constructor tiene que estar adaptado, remember).

```
//4. guardar
vehiculos.put(matricula, v);
```

Y lo guardamos en el HashMap.

Boton mostrar.
Quiero hacer incapié en este componente porque la forma de recorrer el HashMap a través del listener me ha resultado curiosa.

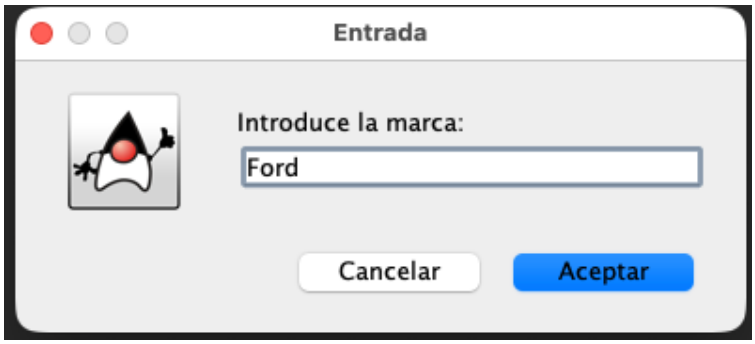
```
// Botón mostrar-------------------

botonMostrar.addActionListener(e->{
    String resultado = "";
    for(Vehiculo v: vehiculos.values()){
        resultado += v + "\n";
    }
    JOptionPane.showMessageDialog(ventana,resultado);
});
```

Cuando el user hace click en el boton, se recorren todos los vehículos guardados y se juntan en un texto para mostrarlos en una ventana. El for recorre uno a uno todos los vehiclos del HashMap y en cada vuelta guarda uno en la variable 'v' para poder usarlo. El caracter \n sirve para hacer un salto de línea, es decir, que dada vehíclo se muestre en una línea distinta.
Al final JOptionPane muestra a traves de showMessageDialog la ventana con el resultado.

BLOQUE DEL OUTPUT:

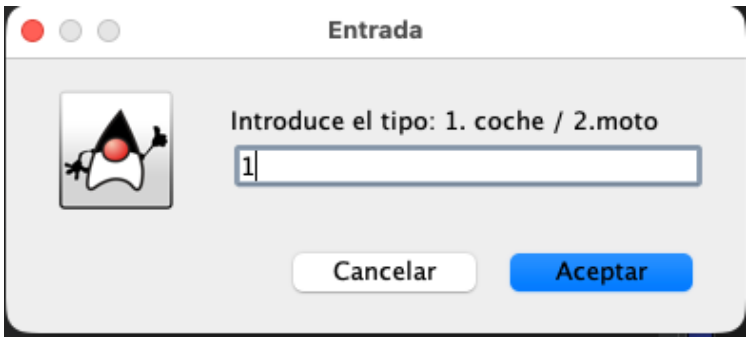
El usuario va a ingresar un vehiculo, por lo tanto hace click en el primer botón Ingresar vehiculo.



Introduce la marca

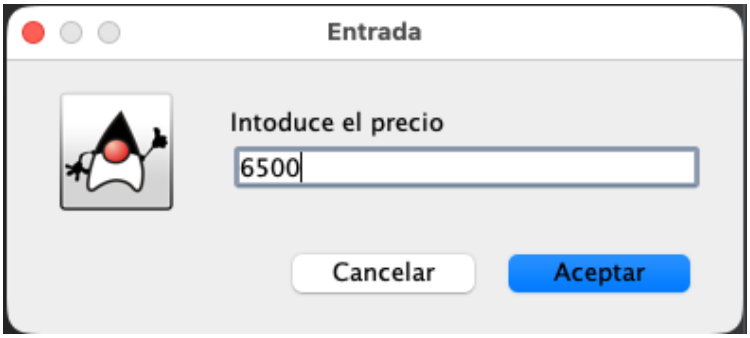
Introduce el modelo

Cilindrada

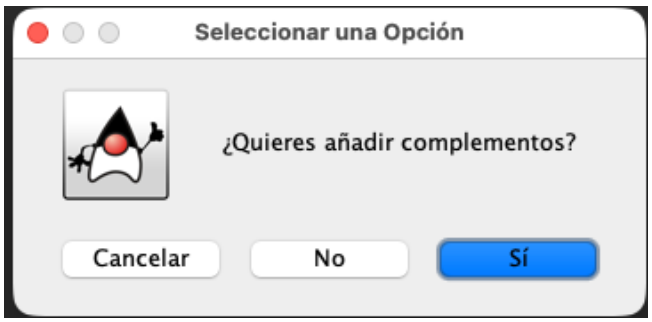


El tipo si es coche o moto

Salta el pop up

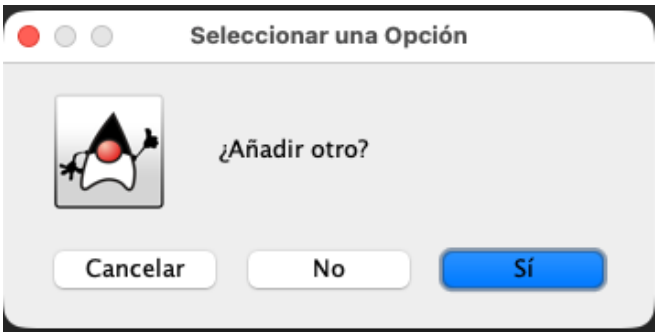


Introduce el precio.

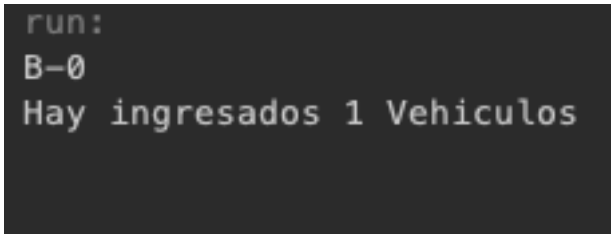


Salta el pop up de los complementos

Introduce un complemento.



Pregunta si queremos otro y lo añadiremos, le pondremos un soporte para el movil. Luego le daremos a NO cuando vuelva a preguntar.



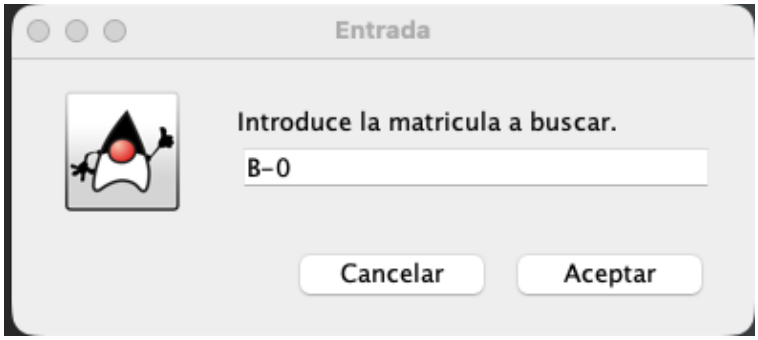
Puse un control por consola para ver el número de mátricula y el ingreso correspondiente.

Ahora a comprobar el resto de acciones.

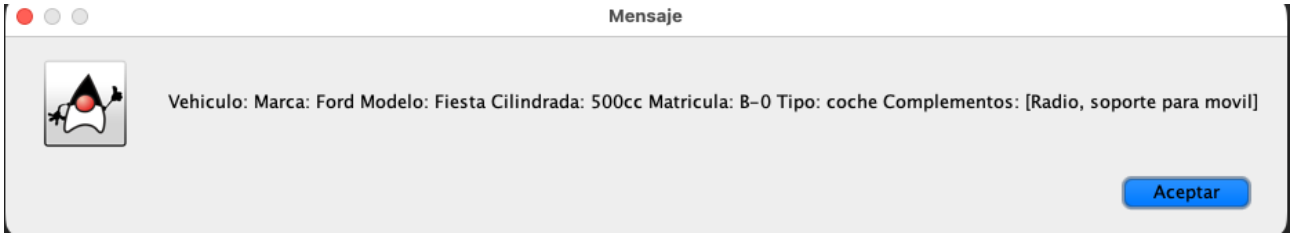
Mostrar vehiculos:

Introduje otro vehiculo más para que se vea que puede mostrar más de uno. Con sus complementos y todo.

Buscar vehiculos:

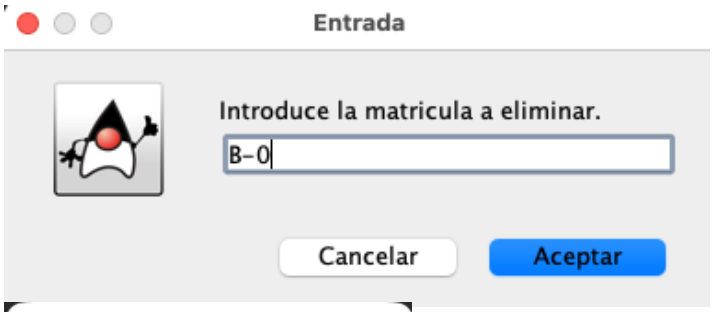


Introducimos la matrícula



Nos devuelve el vehiculo buscado

Eliminar vehículo:



Introducimos la matrícula

El Pop up nos avisa que el vehiculo ha sido eliminado y ahora vamos a comprobarlo con el botón mostrar vehículos.

Finalmente nos muestra el único vehiculo que queda en mi hashmap.

El boton salir:

Utliiza una linea de sintaxis que hace tal que asi:
System.exit(0);
Seguidamente salta un popup:

Popup final.

CONCLUSIONES:

Este trabajo me ha encantado porque he podido ver en acción todo lo aprendido hasta ahora con una interfaz gráfica y la verdad es que es muy gratificante. La experiencia de usar HashMap, ArrayList e interfaz grafica con los componentes, usando los listeners… me he sentido más programador que nunca. Pero como siempre **hay que estudiar mucho.**

Bibliografía consultada y herramientas de aprendizaje: Apuntes Linkia, ChatGPT.